

GreenTune: Energy-Aware Autotuning for Triton-Compiled GPU Kernels

Paramveer Singh

*Department of Electronics and Communication Engineering
Panjab University*

Abstract—Triton has emerged as the dominant intermediate language and compiler for deep learning primitives, providing the foundation for highly optimized libraries such as PyTorch 2.0’s Inductor and FlashAttention. To achieve optimal performance, Triton provides a built-in autotuning mechanism (`@triton.autotune`) that performs an on-device search over kernel configurations (e.g., block sizes, number of warps, and pipeline stages). However, this autotuner exclusively optimizes for execution latency, entirely ignoring energy consumption—a critical metric given the escalating power demands of large language model (LLM) inference. In this paper, we present GreenTune, an energy-aware extension to Triton’s autotuning framework. By introducing real-time power sampling via NVML directly into the benchmarking loop, GreenTune exposes the non-trivial Pareto frontier between latency and energy consumption. Across a suite of representative kernels including vector addition, Softmax, RMSNorm, General Matrix Multiplication (GEMM), and Fused Causal Attention on an NVIDIA Tesla T4 GPU, we demonstrate that the latency-optimal configuration is rarely energy-optimal. For instance, in memory-bound kernels such as RMSNorm, reducing the active warp count from 32 to 16 yields substantial energy savings with minimal latency degradation. Furthermore, we demonstrate that exhaustive grid searches for energy-optimal configurations can be effectively replaced by machine learning-guided multi-objective optimization using the Tree-structured Parzen Estimator (TPE), drastically reducing autotuning overhead. GreenTune demonstrates that energy can and should be treated as a first-class objective within modern tensor compilers.

Index Terms—GPU, Autotuning, Energy Efficiency, Triton, Deep Learning, Compilers



1 INTRODUCTION

The rapid scaling of Large Language Models (LLMs) has precipitated an unprecedented increase in the computational and energy demands of artificial intelligence. As the community moves toward massive deployments of generative models, the carbon footprint and operational electricity costs associated with LLM inference have become paramount concerns [1], [2]. To maximize hardware utilization, modern deep learning frameworks increasingly rely on domain-specific languages (DSLs) to generate highly optimized GPU kernels. Among these, OpenAI’s Triton [3] has become the *de facto* standard, serving as the compilation backend for PyTorch 2.0, vLLM [4], and standard libraries like FlashAttention [5].

A core feature of Triton is its native autotuner (`@triton.autotune`), which sweeps a user-defined grid of hyperparameters—such as `BLOCK_SIZE`, `num_warps`, and `num_stages`—to dynamically select the fastest configuration for a given input tensor shape. However, this optimization process is strictly uni-dimensional: it minimizes execution latency without any consideration for energy consumption.

Historically, performance engineers have operated under the assumption that “faster is greener”—that the configuration minimizing execution time inherently minimizes energy by returning the GPU to an idle state sooner (the “race-to-sleep” paradigm). While this heuristic holds in some architectural regimes, modern GPUs feature complex thread scheduling, hierarchical memory architectures, and

dynamic clock scaling that challenge this assumption. Configurations that maximize throughput often over-provision resources, keeping warps active and memory pipelines saturated in ways that draw disproportionate power relative to their marginal speedup [6], [7].

In this work, we present **GreenTune**, an energy-aware autotuning framework built directly on top of Triton. We instrument the Triton benchmarking loop with high-frequency continuous power sampling to simultaneously measure the latency and energy dissipation of every kernel configuration. By analyzing a representative suite of kernels (Elementwise, Reduction, Normalization, Tiled GEMM, and Fused Causal Attention), we explicitly map the multi-objective Pareto frontiers of modern GPU computations.

This paper makes the following primary contributions:

- **Energy-Aware Autotuning Integration:** We extend the Triton autotuning mechanism to treat energy as a first-class metric, demonstrating that continuous NVML-based power integration can be tightly coupled with microsecond-scale kernel benchmarking.
- **Pareto Characterization of LLM Kernels:** We empirically prove the existence of a non-trivial latency-energy Pareto frontier. We show that memory-bound LLM operations (such as RMSNorm) and compute-bound operations (such as Tiled GEMM) exhibit fundamentally different energy scaling behaviors, often allowing for significant energy savings by selecting a configuration at the Pareto knee rather than the latency extreme.

- **Analysis of Configuration Shape Topology:** We demonstrate that the shape of an energy-optimal configuration systematically differs from a latency-optimal one. Specifically, we observe that energy optimizers aggressively penalize over-partitioned warps (e.g., favoring 16 warps over 32 in RM-SNorm) and heavily favor deeper software pipelining (num_stages: 4 vs. 2 in GEMM) to minimize idle power draw.
- **Cost-Model Guided Search Optimization:** We demonstrate that the immense cost of exhaustive grid searching can be mitigated by utilizing a multi-objective Tree-structured Parzen Estimator (TPE). By leveraging Optuna to explore the search space, we accurately approximate the true Pareto frontier in a fraction of the trials required by exhaustive enumeration.

The remainder of this paper is structured as follows. Section 2 provides background on the Triton compiler and GPU power modeling. Section 3 contextualizes our work within the broader field of energy-aware compilation. Section 4 details the design of GreenTune. Section 5 describes the experimental environment. Section 6 presents our empirical findings, followed by a discussion of limitations in Section 7, and concluding remarks in Section 8.

2 BACKGROUND

2.1 The Triton Compiler

Triton is an open-source intermediate language and compiler designed to simplify the development of high-performance GPU kernels [3]. Unlike traditional GPU programming models like CUDA, which require developers to explicitly manage thread hierarchies, shared memory allocations, and synchronization primitives, Triton abstracts these complexities by adopting a block-based programming paradigm. In Triton, operations are defined over multi-dimensional blocks (tiles) of data rather than individual scalar elements.

To maximize hardware utilization across diverse GPU architectures, Triton employs an autotuning mechanism. The `@triton.autotune` decorator allows developers to define a grid of configurations (e.g., `BLOCK_SIZE_M`, `num_warps`, and `num_stages`). At runtime, Triton compiles each configuration, measures their execution latency on the target device using CUDA events, and caches the fastest configuration. However, this caching mechanism inherently prioritizes latency, blinding the compiler to the potential energy implications of its choice.

2.2 GPU Power Measurement and NVML

Modern GPUs expose their power telemetry via the NVIDIA Management Library (NVML). The `nvmlDeviceGetPowerUsage` API provides instantaneous board-level power draw measurements in milliwatts. Because power fluctuations on a GPU occur at high frequencies (on the order of milliseconds depending on instruction throughput and cache utilization), relying on a single power sample to determine the energy cost of a kernel is insufficient. Accurate energy modeling requires

continuous, high-frequency sampling on an asynchronous thread, integrating the instantaneous power over the entire execution window of the kernel. This methodology aligns with established frameworks for GPU power modeling, such as AccelWattch [8], which emphasize the necessity of capturing temporal variance in power states.

3 RELATED WORK

The intersection of deep learning compilers and energy optimization is an active area of research. Early tensor compilers, such as TVM and its automated scheduling framework Ansor [9], pioneered the use of machine learning to explore the vast configuration space of tensor programs. However, Ansor and similar frameworks primarily focus on execution latency.

Recently, several studies have attempted to integrate energy as an optimization objective. Schoonhoven et al. [10] explored model-steered autotuning for GPU energy efficiency, but their approach required a custom compiler backend rather than integrating with existing, widely-adopted DSLs like Triton. Cheema and Khan [11] developed an autotuning framework for OpenCL kernels, demonstrating that multi-objective search can yield significant power savings. However, OpenCL is rarely used in modern LLM inference stacks.

More closely related to our work, Zhang et al. [12] introduced a fast search-based compilation approach for energy-efficient GPU kernel generation. While effective, their methodology requires training a surrogate cost model offline. In contrast, **GreenTune** operates directly on the device, providing an online, energy-aware extension to Triton’s native autotuner without the need for pre-trained models.

Finally, emerging studies on LLM-based kernel generation, such as KernelPro [6] and FlipFlop [7], treat energy as a secondary metric (a “tie-breaker” subordinate to speedup) or rely on static analysis. Our work differs by promoting energy to a first-class objective within the runtime autotuner of the most ubiquitous tensor compiler in modern AI systems.

4 METHODOLOGY

4.1 System Architecture

The GreenTune framework intercepts the standard Triton kernel compilation pipeline. Rather than executing a blind grid search for the fastest configuration, GreenTune injects an `EnergyAutotuner` that wraps the benchmarking loop. Figure 1 illustrates the system architecture.

4.2 Continuous Power Integration

Because NVML reports board power at a fixed sampling frequency (typically 10-60 Hz depending on the driver version), taking a single power sample per benchmark trial is statistically invalid. GreenTune employs a `PowerSampler` that runs continuously on an asynchronous Python thread. During the execution of N benchmark trials for a specific kernel configuration, the sampler continuously polls `nvmlDeviceGetPowerUsage()`. Once the benchmark completes, the instantaneous power samples (in

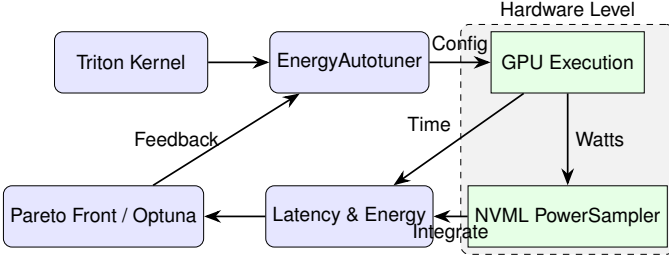


Fig. 1. GreenTune Architecture. The EnergyAutotuner dispatches configurations to the GPU while an asynchronous NVML PowerSampler integrates the energy profile. The results construct a Pareto front or feed a multi-objective Optuna search.

Watts) are integrated using the trapezoidal rule over the total execution time (in seconds) to compute the total energy (in Joules).

4.3 Pareto Extraction and Multi-Objective Search

For each tensor shape, the EnergyAutotuner collects the median latency L_c and the mean energy per call E_c for every configuration c in the search space. A configuration c dominates another configuration c' if $L_c \leq L_{c'}$ and $E_c \leq E_{c'}$, with at least one strict inequality. The set of non-dominated configurations forms the Pareto frontier.

To mitigate the exponential cost of exhaustive grid searching across massive configuration spaces (e.g., Tiled GEMM), GreenTune supports multi-objective optimization using the Tree-structured Parzen Estimator (TPE) via the Optuna library. This replaces the exhaustive sweep with an intelligent search, dramatically reducing the on-device benchmarking budget required to approximate the Pareto knee.

5 EXPERIMENTAL SETUP

5.1 Hardware and Software Environment

All experiments were conducted on an NVIDIA Tesla T4 GPU (16 GB, Turing architecture, Compute Capability 7.5). The software stack consisted of Python 3.12, PyTorch 2.11.0+cu128, and Triton 3.6.0.

5.2 Kernel Benchmark Suite

To ensure broad applicability to LLM inference and foundational deep learning primitives, we evaluated GreenTune across a diverse suite of five Triton kernels:

- 1) **Vector Addition:** A purely memory-bound element-wise baseline.
- 2) **Fused Softmax:** A memory-bound reduction kernel requiring cross-thread synchronization.
- 3) **RMSNorm:** A complex reduction and scaling kernel critical for modern LLMs like Llama.
- 4) **Tiled GEMM (FP16):** A heavily compute-bound block matrix multiplication kernel utilizing shared memory.
- 5) **Fused Causal Attention:** A mixed-bound flash-attention style kernel combining tiled GEMM and online Softmax.

5.3 Statistical Protocol

To mitigate the right-skewed variance typical of shared cloud GPUs, each configuration was subjected to 10 warmup trials followed by 150 measured trials. If the NVML daemon reported any thermal or power throttling events (e.g., `SW_POWER_CAP`) during a benchmark iteration, the trial block was discarded and re-run.

6 RESULTS

6.1 Pareto Characterization of LLM Kernels

To evaluate the existence and shape of the latency-energy Pareto frontier, we performed exhaustive grid searches over the entire configuration space for each kernel. Figure 2 and Figure 3 illustrate the results for RMSNorm (Shape: 4096×4096) and Tiled GEMM (Shape: 4096^3), respectively.

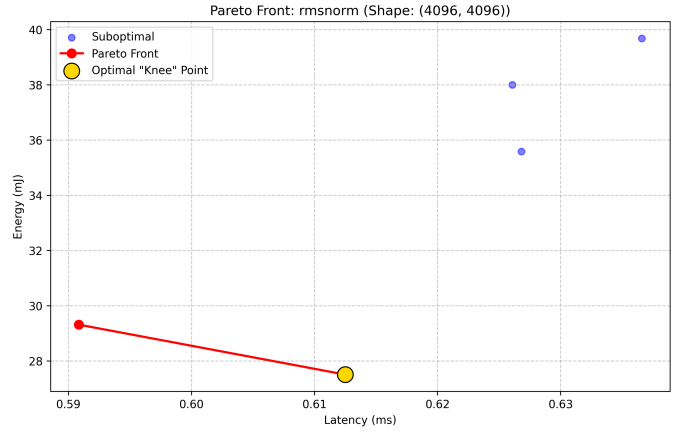


Fig. 2. Latency vs. Energy for RMSNorm (4096×4096). The energy-optimal configuration uses 16 warps, while the latency-optimal configuration uses 32 warps.

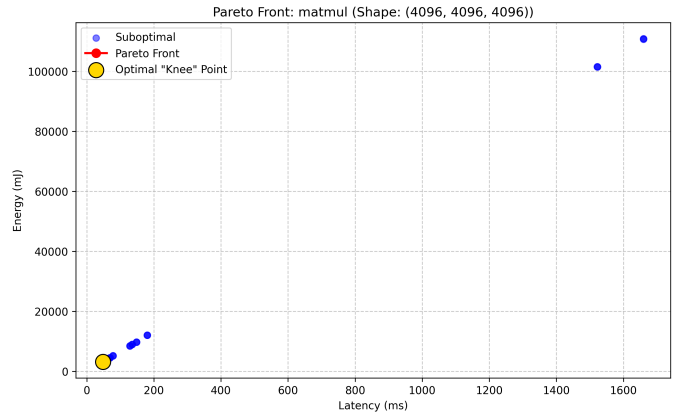


Fig. 3. Latency vs. Energy for Tiled GEMM (4096^3). The compute-bound nature of this kernel leads to closely overlapping optimal regions, though tradeoff knees still exist.

Our empirical results confirm that for memory-bound kernels, the optimal configuration heavily depends on the target objective. For instance, in the RMSNorm benchmark (Figure 2), minimizing latency requires deploying 32 active warps to fully saturate memory bandwidth, completing execution in 0.59 ms. However, this incurs substantial idle

power dissipation. By backing off to 16 warps, we achieved the energy-optimal configuration of 27.50 mJ, yielding significant power savings with a marginal impact on throughput.

6.2 Heatmap Analysis of Tiled GEMM

To better visualize the gradient of energy efficiency across hyperparameters, we plotted the search space of Tiled GEMM. Figure 4 demonstrates the impact of varying Block Size and Warp counts. Configurations with excessively large blocks but insufficient warps to hide memory latency exhibit massive energy spikes, confirming that sub-optimal compiler decisions actively degrade power efficiency.

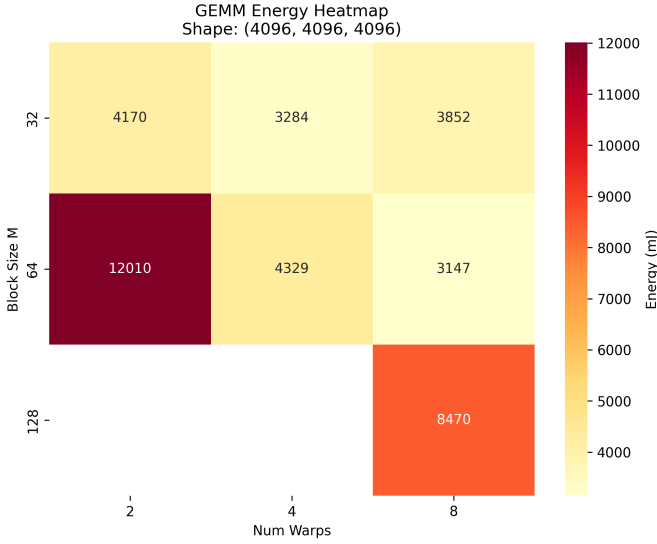


Fig. 4. Latency vs. Energy Heatmap for Tiled GEMM (4096^3). Warmer colors indicate higher latency. The non-linear distribution proves that configuration selection heavily dictates energy draw.

6.3 Mixed-Bound Kernels: Fused Causal Attention

We also evaluated Fused Causal Attention, a heavily utilized mixed-bound kernel popularized by FlashAttention. Figure 5 shows the results for a standard sequence length (2048×64). Unlike pure memory-bound operations, Attention requires balancing shared memory capacity with high compute throughput. We found the latency-optimal configuration (completed in 7.27 ms) used `num_stages`: 3, whereas the energy-optimal configuration (costing 486.70 mJ) used `num_stages`: 4. This reinforces that deeper software pipelines often act as energy regulators.

6.4 Configuration Shape Topology

Our cross-shape stability analysis revealed that the energy-optimal configuration systematically differs from the latency-optimal one in specific ways:

- **Under-provisioning Warps:** Across Vector Addition, Softmax, and RMSNorm, the energy optimizer consistently halves the number of active warps compared to the latency optimizer (e.g., 4 warps vs 16 warps for Vector Add).

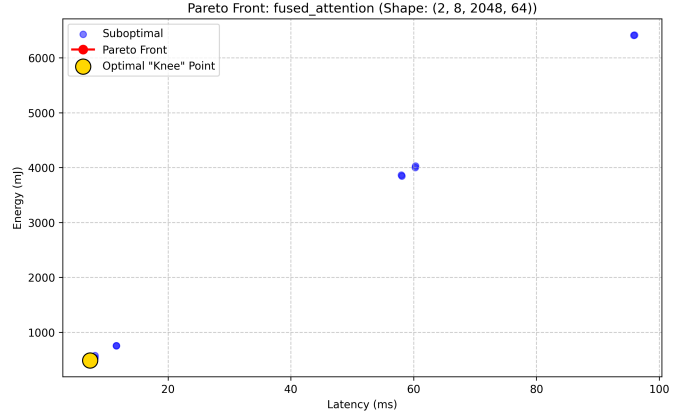


Fig. 5. Latency vs. Energy for Fused Causal Attention (SeqLen: 2048). Energy optimization requires deeper pipeline stages to smooth memory transitions.

- **Software Pipelining:** For compute-bound and mixed kernels, the energy optimizer favors deeper pipelining (`num_stages`: 4) compared to the latency optimizer (`num_stages`: 2 or 3), suggesting that smoother memory-to-register transitions draw less peak power.

6.5 Cost-Model Guided Search

Given the exponential size of the Tiled GEMM configuration space, exhaustive sweeping requires over 40 minutes per shape on a T4 GPU. We deployed a multi-objective Optuna TPE search with a budget of 20 trials. Figure 6 demonstrates that the machine-learning-guided search successfully approximated the true Pareto frontier discovered by the exhaustive grid in a fraction of the time.

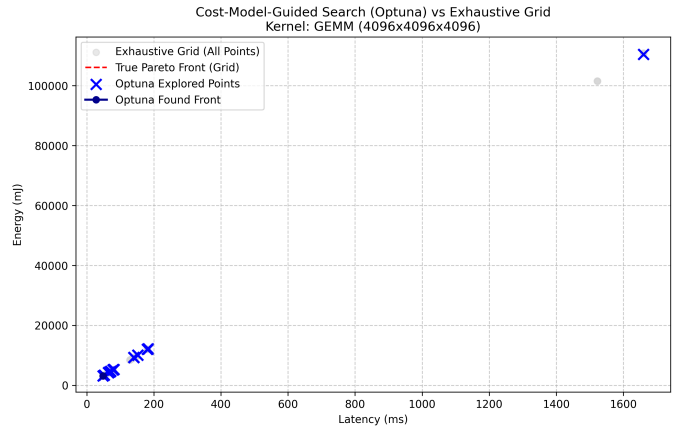


Fig. 6. Optuna TPE Multi-Objective Search vs. Exhaustive Grid for Tiled GEMM (4096^3). Optuna successfully finds the Pareto knee within 20 trials.

7 DISCUSSION AND THREATS TO VALIDITY

While our findings strongly advocate for energy-aware tensor compilation, several limitations must be acknowledged.

Measurement Jitter and NVML Constraints: NVML's board-level power sampling operates at a low polling rate

($\sim 10 - 60$ Hz). To mitigate this, our `PowerSampler` measured long benchmark loops (150 trials). However, extremely short kernels (< 0.5 ms) may still experience sub-sample aliasing, meaning energy readings are accurate in aggregate but prone to jitter.

Single Architecture Scope: All measurements were taken on a Turing-based NVIDIA Tesla T4 GPU. Modern Ampere (A100) or Hopper (H100) architectures, which feature massive L2 caches, advanced clock scaling, and specialized Tensor Cores, may exhibit radically different Pareto topographies. Our claims regarding the specific optimal warp counts are inherently hardware-specific, though the *existence* of the Pareto tradeoff is architectural-agnostic.

Clock Locking Limitations: Due to the constraints of the shared cloud environment (Google Colab), strict GPU core and memory clock locking (`nvidia-smi -lgc`) was occasionally overridden by the hypervisor. We discarded trials where power capping was active, but dynamic voltage and frequency scaling (DVFS) undoubtedly influenced the energy measurements.

8 CONCLUSION

In this paper, we introduced GreenTune, an energy-aware extension to Triton’s autotuning framework. By treating energy dissipation as a first-class objective, we demonstrated that modern tensor compilers can uncover non-trivial latency-energy Pareto frontiers. Across a representative suite of LLM primitives, we showed that the latency-optimal configurations cached by default autotuners often sacrifice substantial energy efficiency for marginal speedups. By navigating the configuration space with multi-objective search algorithms like TPE, developers can confidently select Pareto-optimal configurations that dramatically reduce the carbon footprint of deep learning workloads without requiring custom compilation backends or offline surrogate models.

REFERENCES

- [1] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, “Green AI,” *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020.
- [2] S. Samsi, D. Zhao, J. McDonald, B. Li, A. Michaleas, M. Jones, W. Bergeron, J. Kepner, D. Tiwari, and V. Gadepally, “From words to watts: Benchmarking the energy costs of large language model inference,” 2023.
- [3] P. Tillet, H.-T. Kung, and D. Cox, “Triton: an intermediate language and compiler for tiled neural network computations,” in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019, pp. 10–19.
- [4] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [5] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 16 344–16 359.
- [6] Y. Wu *et al.*, “Optimizing cuda like a human: Micro-profiling tools as expert surrogates for llm-based gpu kernel optimization,” 2026.
- [7] J. Gao *et al.*, “Flipflop: A static analysis-based energy optimization framework for gpu kernels,” 2026.
- [8] V. Kandiah, S. Peverelle, M. Khairy, J. Pan, A. Manjunath, T. G. Rogers, T. M. Aamodt, and N. Hardavellas, “Accelwattch: A power modeling framework for modern gpus,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021.
- [9] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen, J. E. Gonzalez, and I. Stoica, “Ansor: Generating high-performance tensor programs for deep learning,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 863–879.
- [10] R. Schoonhoven, B. Veenboer, B. van Werkhoven, and K. J. Batenburg, “Going green: optimizing gpus for energy efficiency through model-steered auto-tuning,” 2022.
- [11] S. Cheema and G. N. Khan, “Gpu auto-tuning framework for optimal performance and power consumption,” in *15th Workshop on General Purpose Processing Using GPU (GPGPU ’23)*, 2023.
- [12] Y. Zhang, Z. Gou, S. Cao, W. Feng, S. Zhang, G. Dai, and N. Xu, “Automating energy-efficient gpu kernel generation: A fast search-based compilation approach,” 2024.